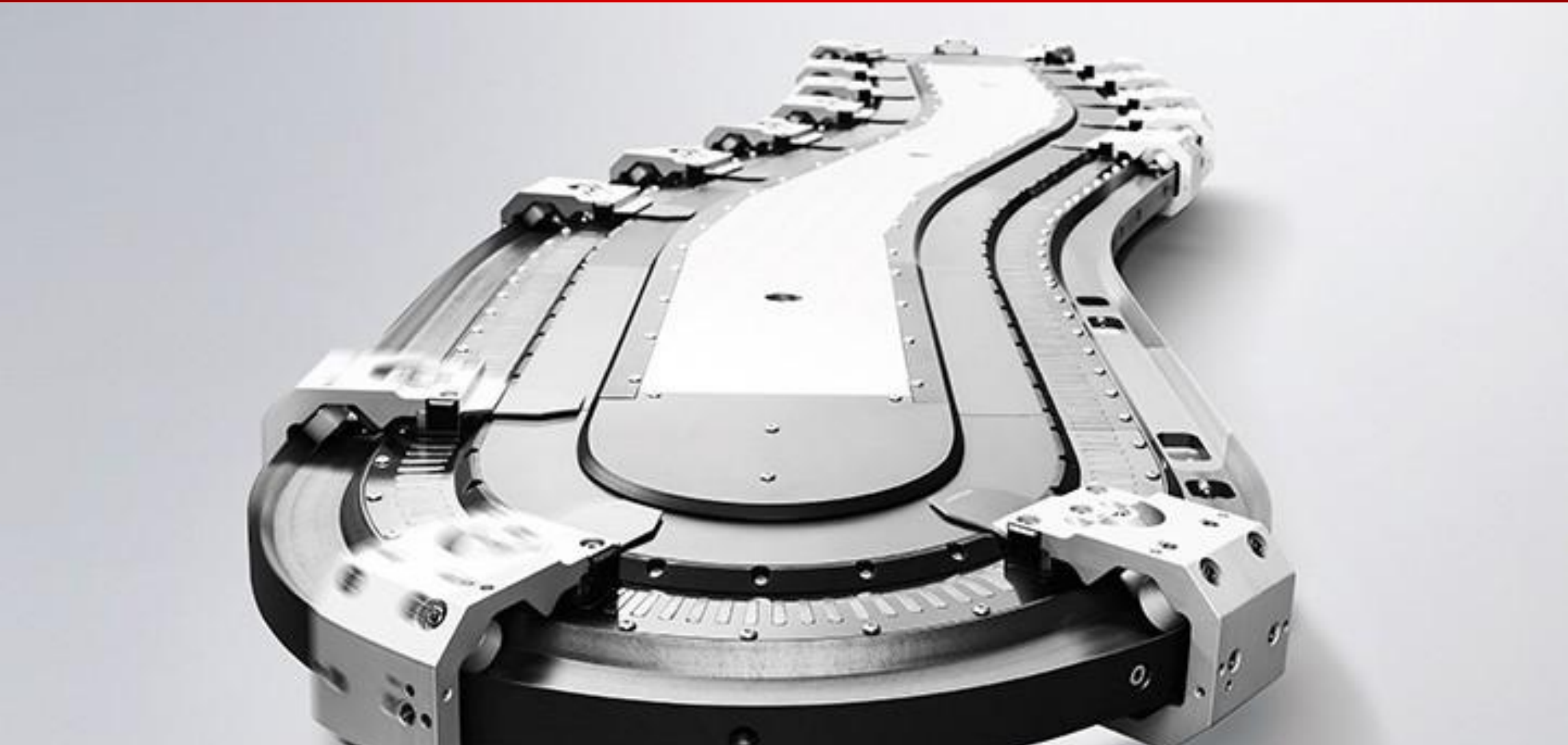
XTS TRANSPORT LAYER – The Collectors

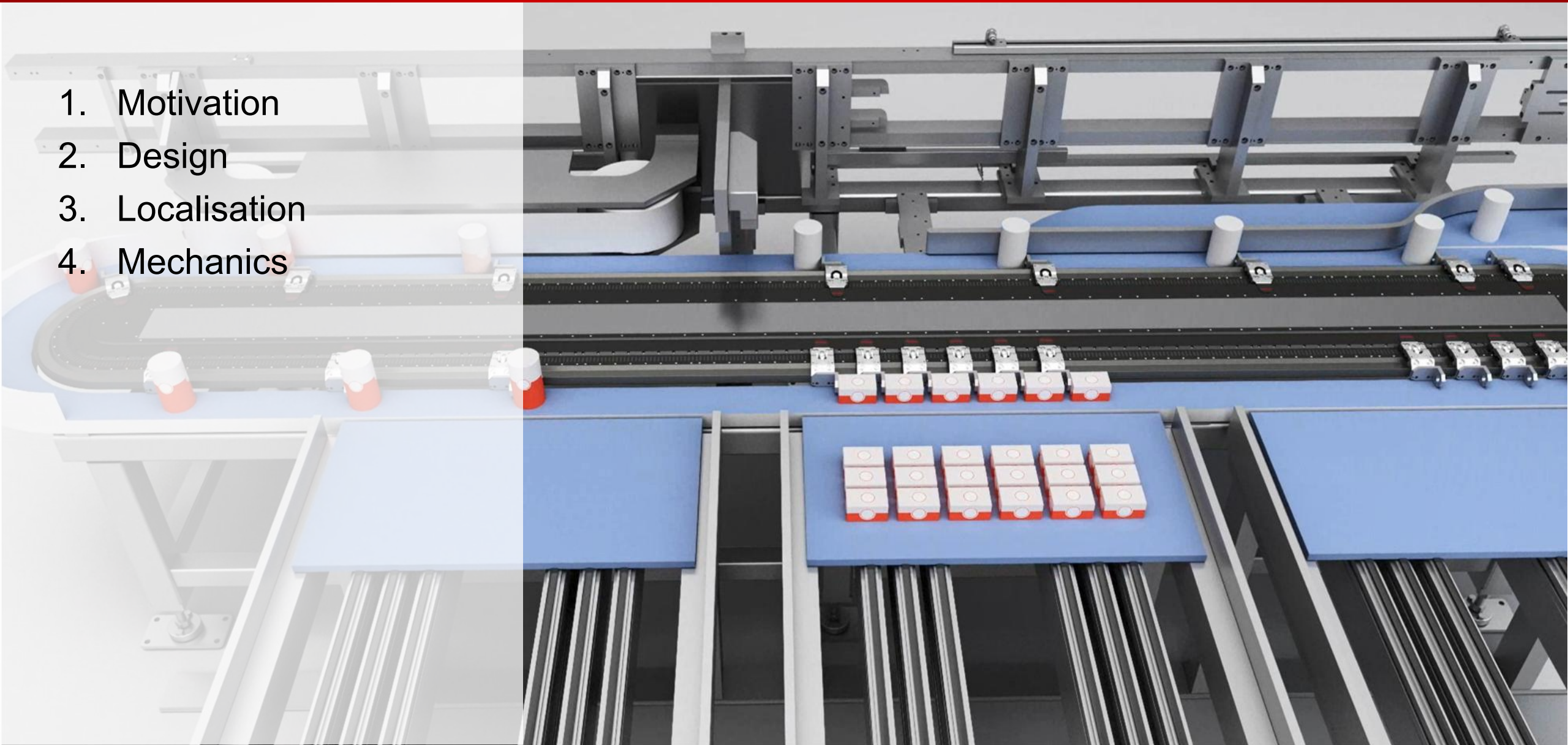
BECKHOFF



XTS TRANSPORT LAYER – The Collectors

BECKHOFF

1. Motivation
2. Design
3. Localisation
4. Mechanics



- **One station one mover.**
  - Parallel stations require a strict structural aggregator to collect and send the handshakes to the base layer of this architecture.
    - **The Collectors.**
  - The Collectors shall be communication agnostic and use the already established software mechanics.
    - **`ST_*_CTRL/ST_*_STATE`** structures for control
    - State machine behaviour shall be similar to the stations' behaviour.
  - The Collectors shall implement an interface to enable composition in higher layers.

- **The Collectors mirror the station-based approach**
  - The Collectors shall be oblivious to the geometry of the stations
    - State aggregator.
    - Cmd deployer.
  - The Collectors shall offer muting of stations in their range
    - The controlling logic retains dynamic R/W access through cycle-by-cycle decision-making regarding whether a station is active.
    - An automated distribution shall be offered.
  - The Collectors shall enable the same handshake routes that are already established in the stations.

- **The Collector decides the target**
  - A Collector shall have **one** target for all its stations.
    - → A Collector aggregates **multiple** stations to route to a single target (m to 1)
  - A Collector shall have many targets.
    - → Collector aggregates **one** station. (1 to m)
      - → Collector mirrors behaviour of a single station.
        - → Collector may work stationary.
        - → Collector may also work as a fly-by distributor.
          - → Distribution to targets may be bitmask controlled

BECKHOFF

- **Station-based approach for the Collectors**

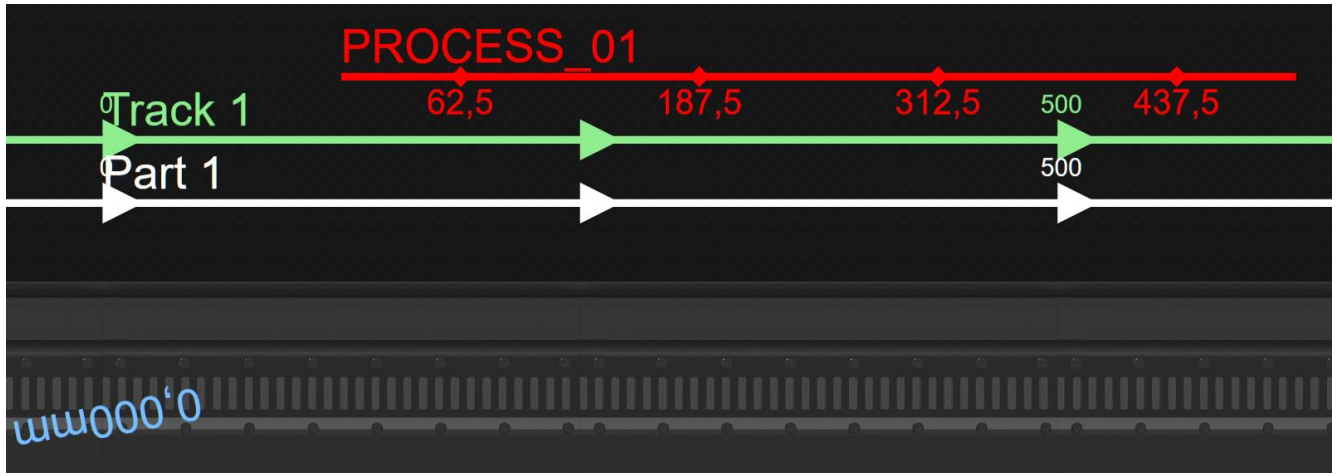
- **Example of a closed loop XTS:**
  - **COLLECTOR** may have multiple stations
    - Process may require a Collector
    - **STATION** may have multiple nests
      - **NEST** is a stop position for a mover



BECKHOFF

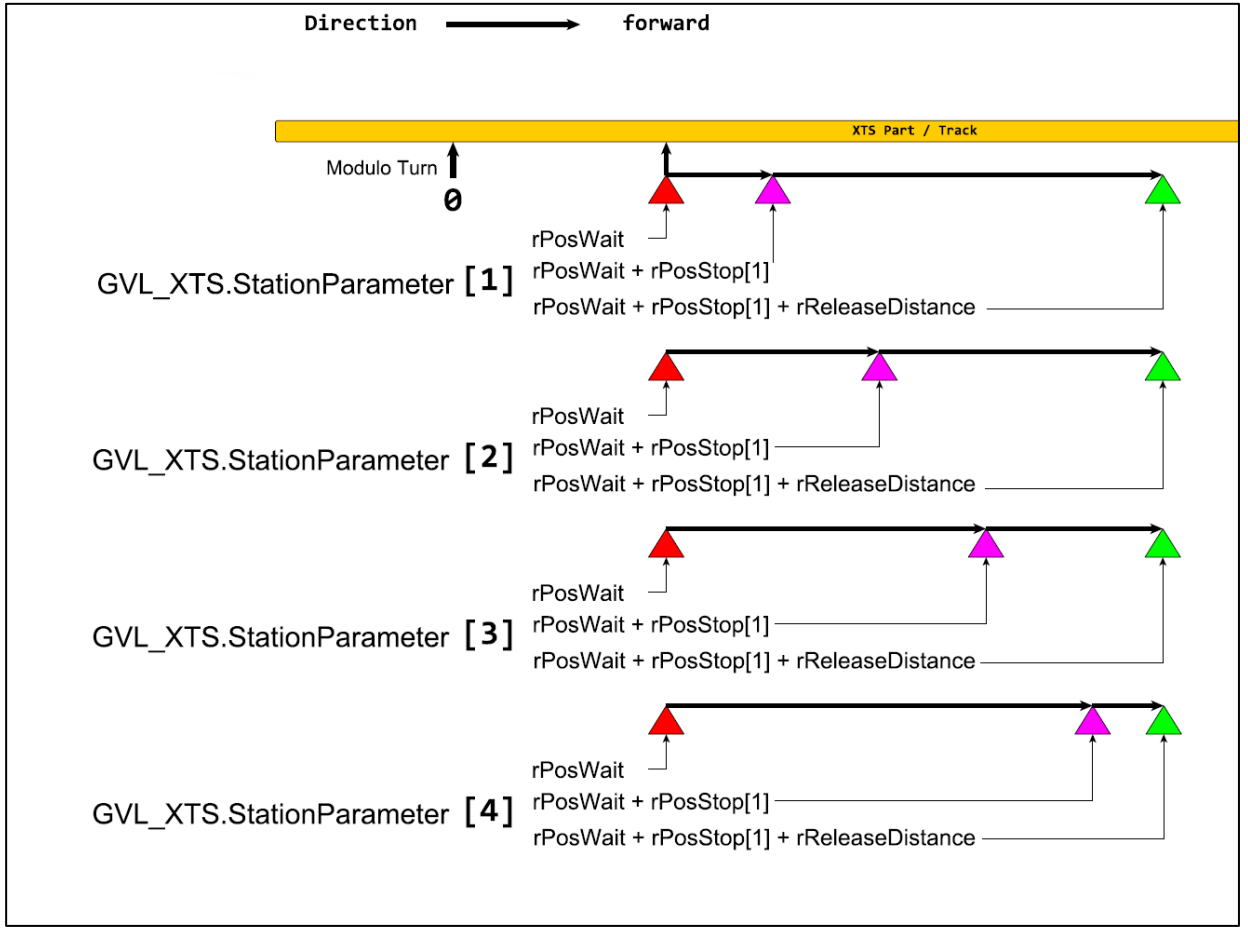
- **Station based approach for the Collectors**

- **PROCESS controls the Collector:**
  - may have one or many stations
  - handshakes stations simultaneously
  - may mute stations
  - Stations in processes may have multiple nests



- **Station based approach for the Collectors**

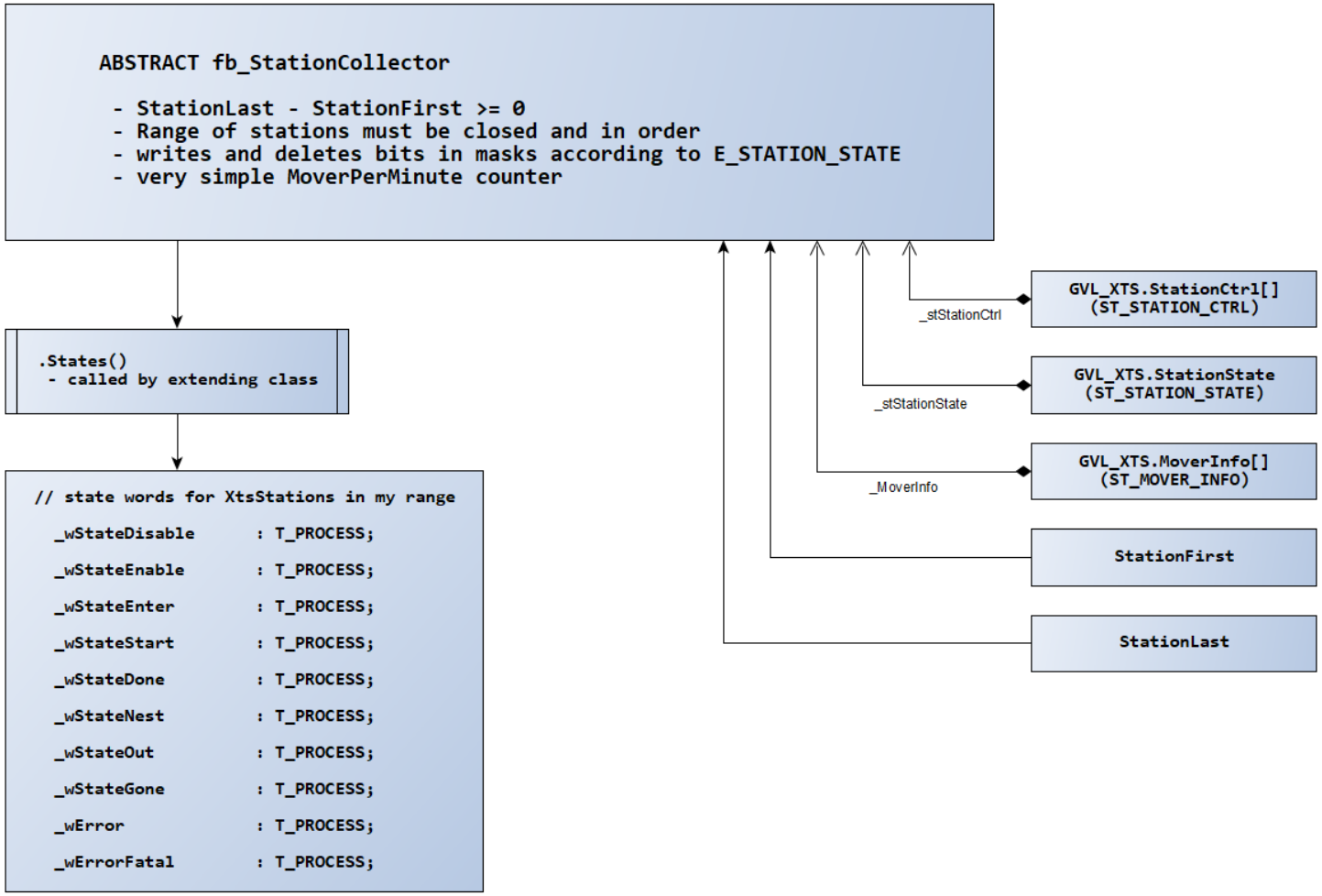
- **COLLECTOR aggregates data:**
  - one or many stations
  - handshakes stations simultaneously
  - Process may mute stations when switching on the Collector.
  - Stations in processes may have multiple nests.
    - In the stations of a Collector the mask for the nests is the same.
      (varying masks possible, if this helps with throughput? doubtful)



BECKHOFF

- **The Collectors:**

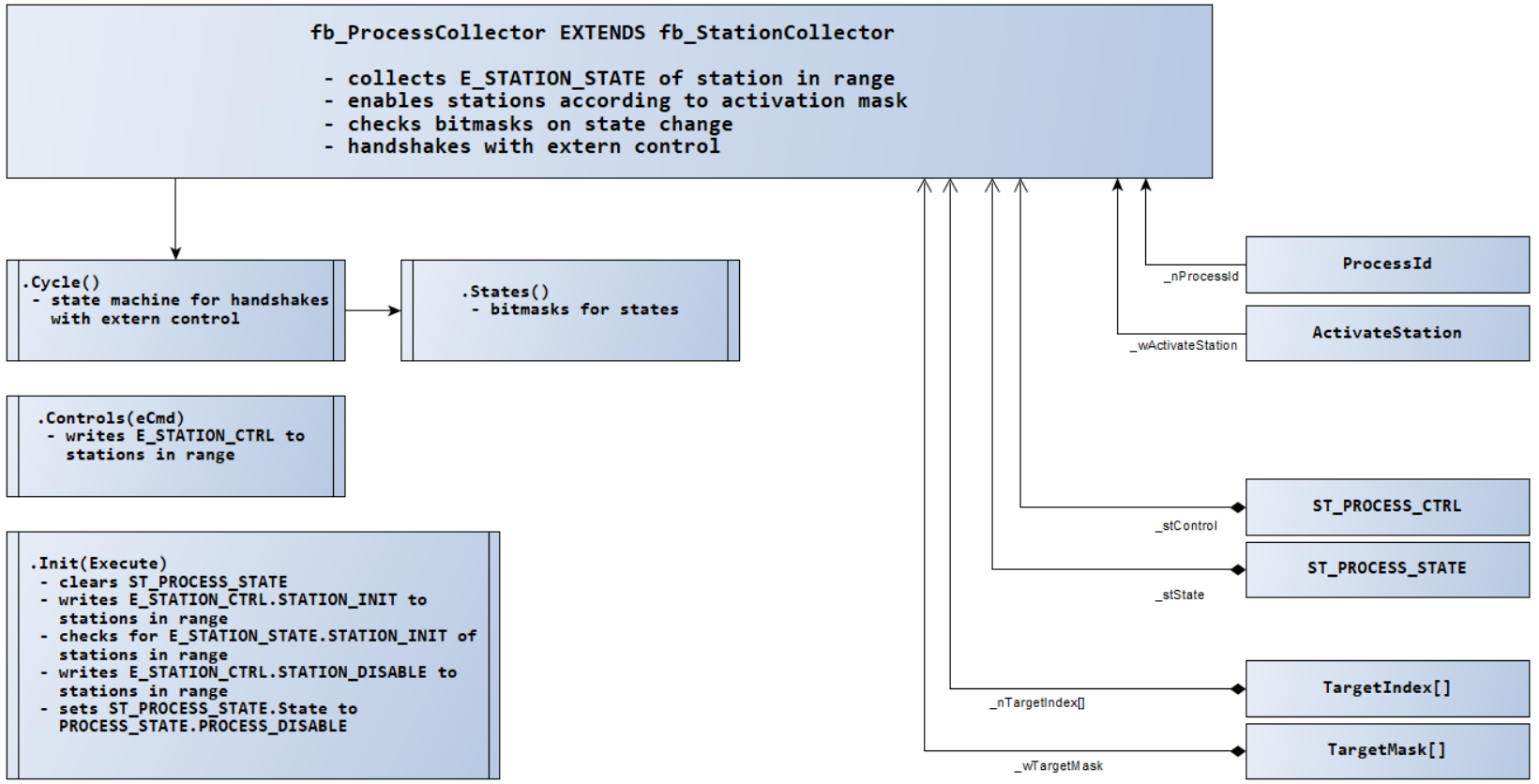
- **StationCollector:**
  - collects information from the stations.



BECKHOFF

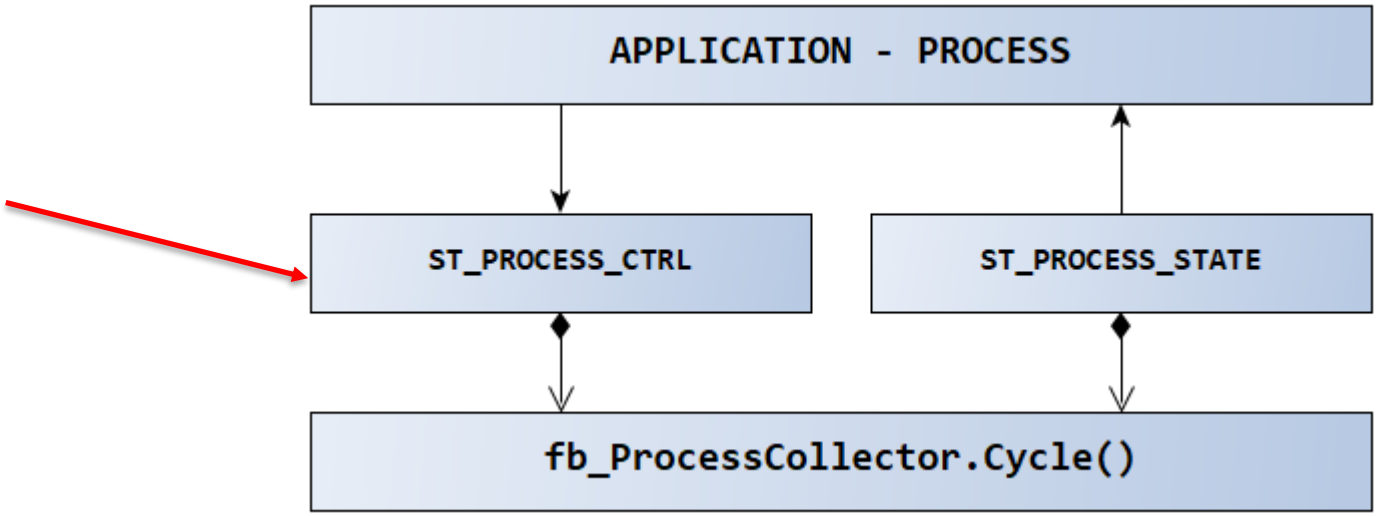
- **The Collectors**

- **ProcessCollector:**
  - commands stations via dedicated structures
  - Checks range of activated stations if their states match the commanding bitmask.



- **The Collectors**

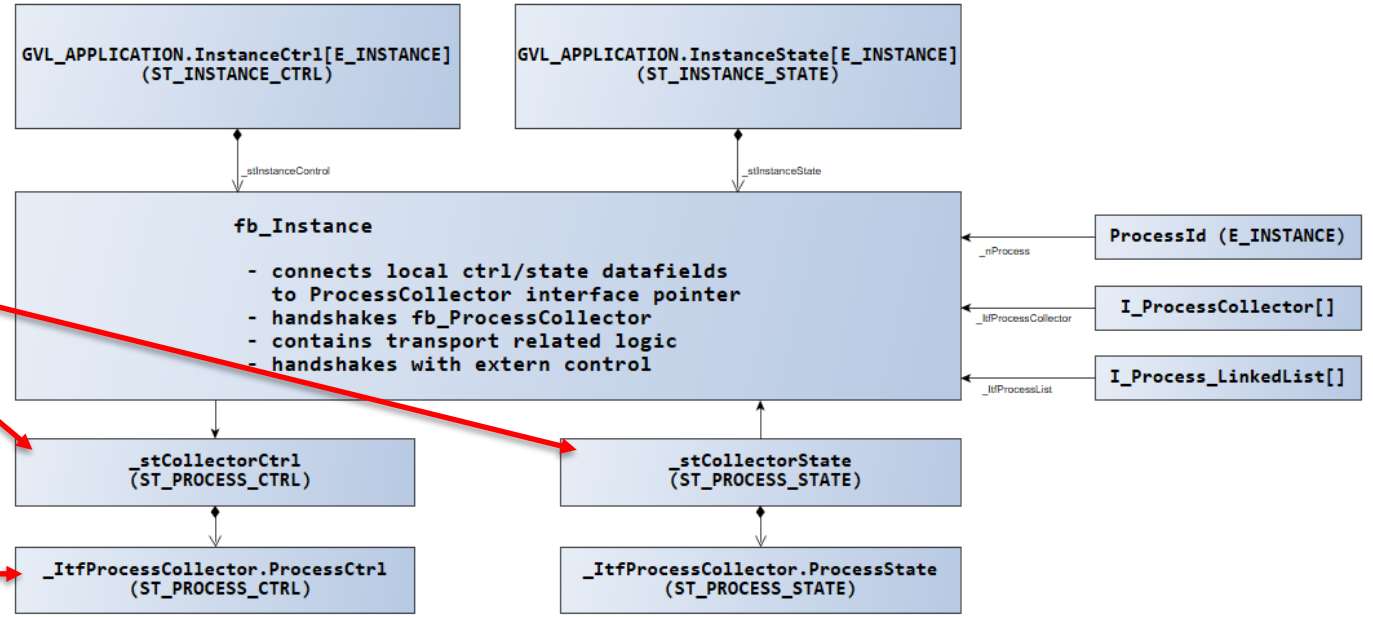
- **ProcessCollector:**
  - **The global composition:**
  - Controlled via global CTRL/STATE data structures.
  - Passed into the collector as reference.
  - You may choose where the structures are placed



BECKHOFF

- **The Collectors and Instances – Collectors first:**

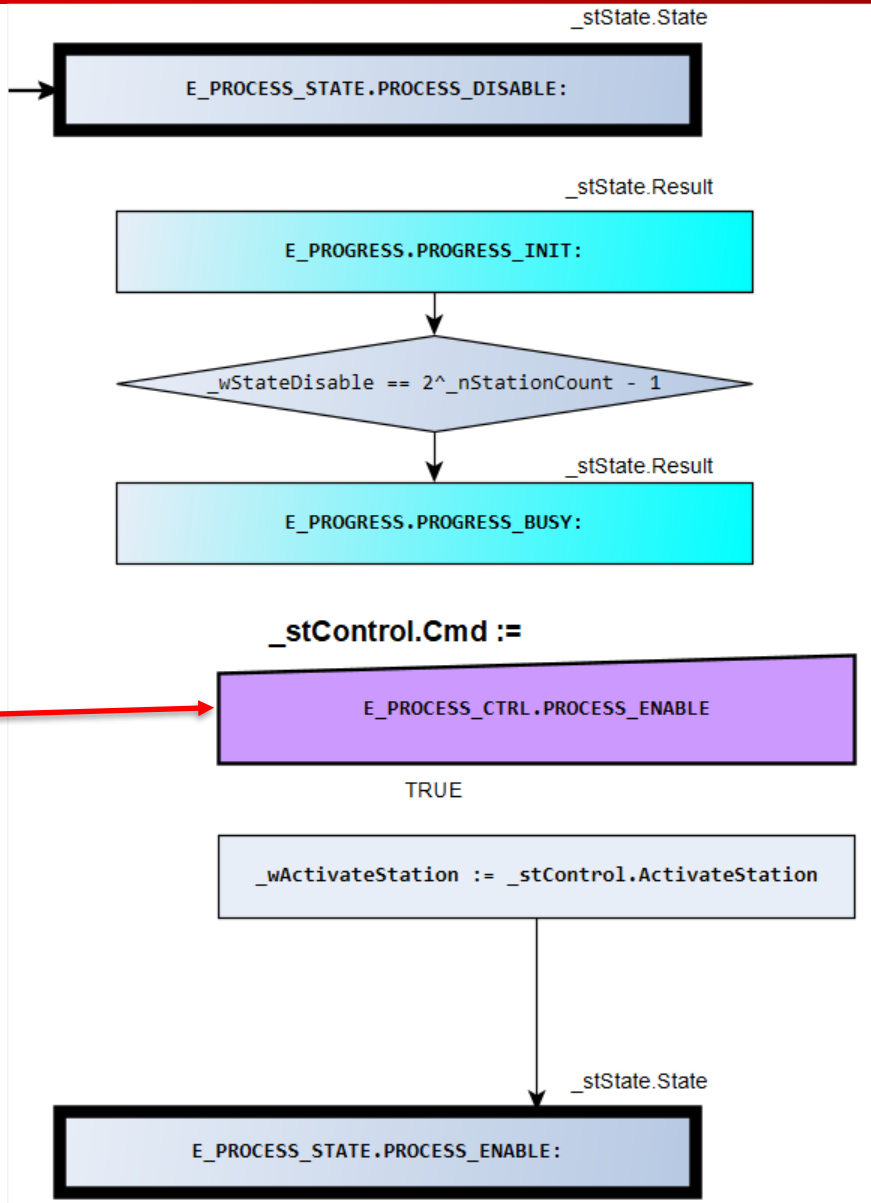
- **ProcessCollector:**
  - **The local composition:**
- Controlled via fb_Instance **ST_PROCESS_CTRL/ST_PROCESS_STATE** data structures.
  - I_ProcessCollector[] is passed into property: fb_Instance.CollectorInterface.
  - **ST_PROCESS_CTRL/ST_PROCESS_STATE** References in fb_ProcessCollector are connected to local instances on startup.
  - ProcessId (E_INSTANCE) enumeration defines place in fb_ProcessCollector[1..MAX_PROCESS]



BECKHOFF

- **The Collectors**

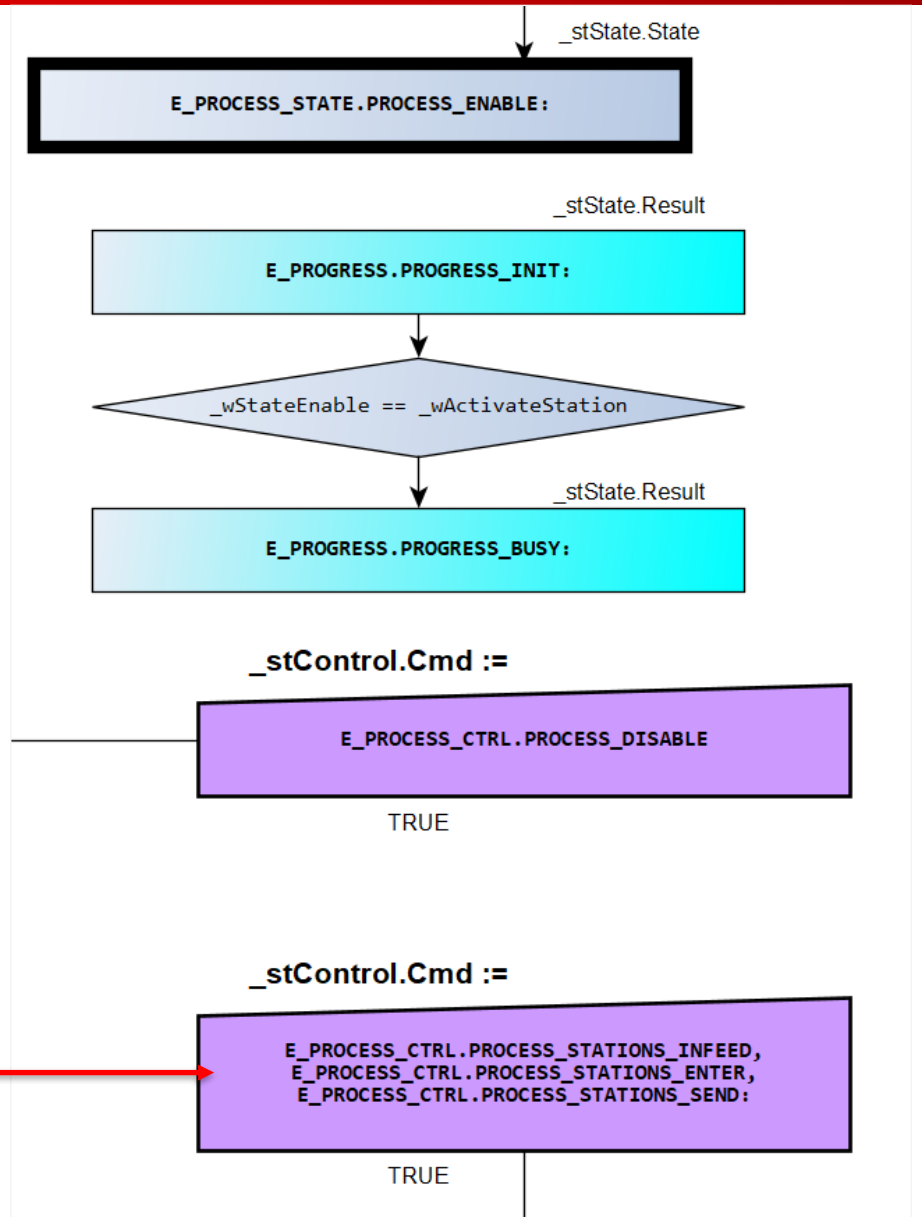
- **ProcessCollector:**
  - Self initializing
    - after initialization,
      the ProcessCollector switches to:
    - PROCESS_DISABLE
  - Must be enabled
    for every lifecycle of the mover in
    station.



BECKHOFF

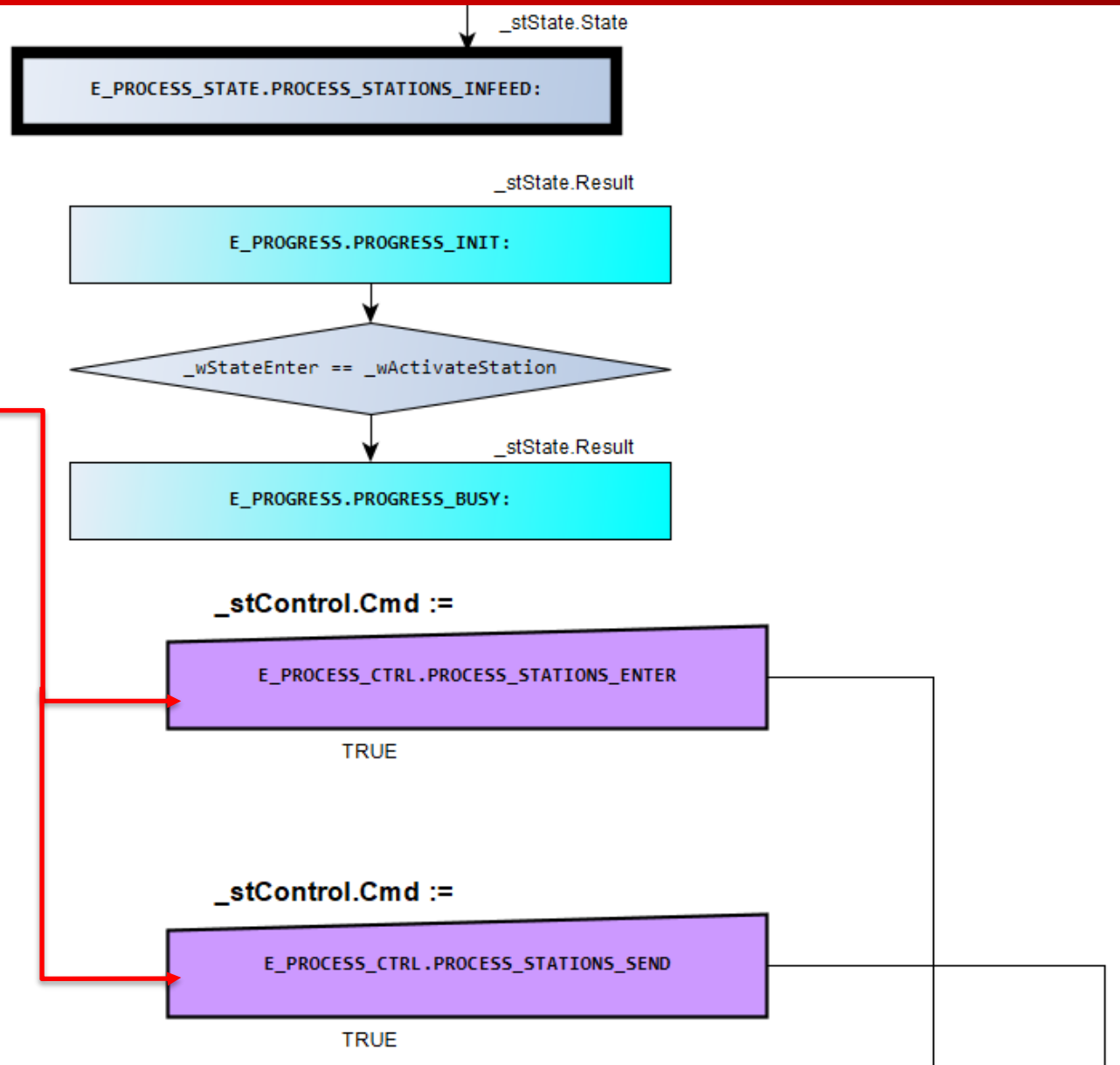
- **The Collectors**

- **ProcessCollector:**
  - FSM behaviour
    - Handshake routes available
      for guiding one or many movers
      through a process,
      the ProcessCollector follows the
      same behaviour as a XtsStation



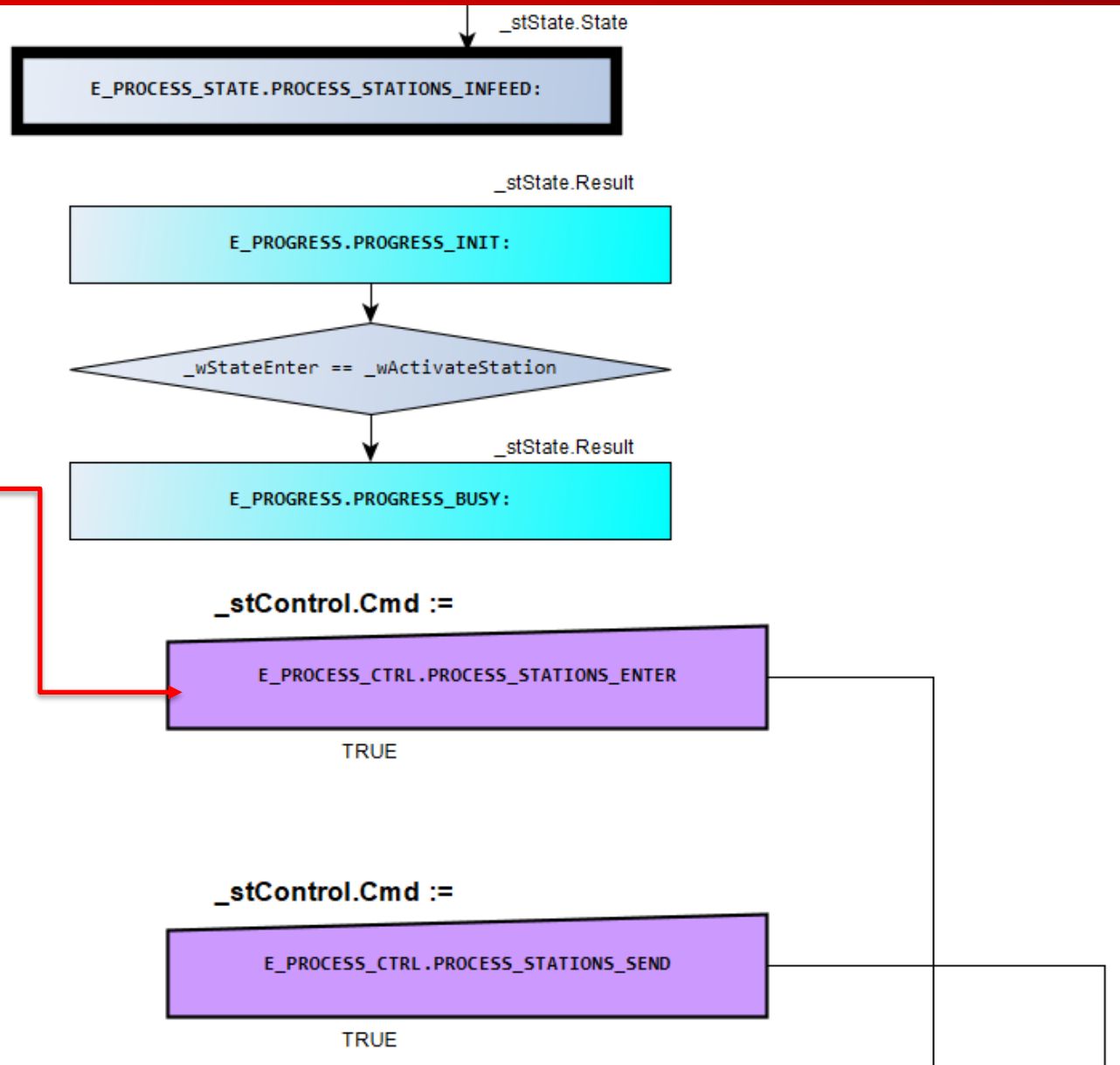
- **The Collectors**

- **ProcessCollector:**
  - FSM behaviour
    - Handshake routes available
      for guiding one or many movers
      through a process,
      the ProcessCollector follows the
      same behaviour as a XtsStation



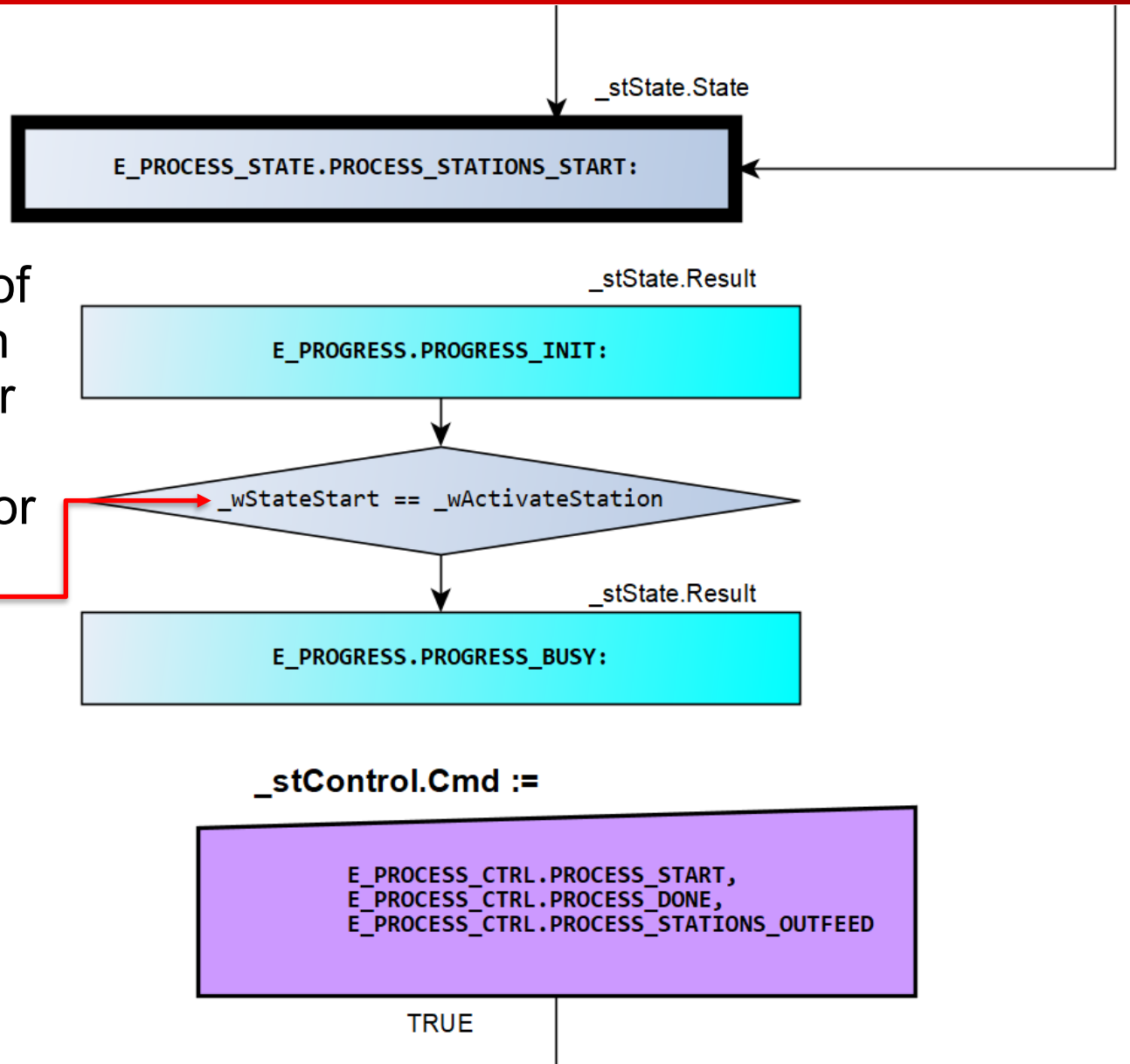
- **The Collectors**

- **ProcessCollector:**
  - Handshake routes available
  - Command for the infeed of the movers into the stations



- **The Collectors**

- **ProcessCollector:**
  - After commanding the infeed of the waiting movers, the station checks the arrival of the mover at PosStop[n] and sets station state which the StationCollector aggregates.
  - Command for the process handshake and the outfeed of the movers out of the stations are accepted once all movers have arrived and are in standstill



BECKHOFF

- **The Collectors - basics**

- **StationCollector:**
  - The base class:
    - Properties

```
FUNCTION_BLOCK ABSTRACT fb_StationCollector
VAR
  {info ' extend/add enum members to E_INSTANCE as you need them '}
  _nProcessId     : E_INSTANCE; // whoami

  _eCheck             : E_PROCESS_CHECK;
  _nStationCount      : UINT;

  _nStationFirst,
  _nStationLast       : UINT; // closed range of XtsStations

  // ctrl / state datafields for extern control
  _stStationCtrl      : REFERENCE TO ARRAY[1..MAX_STATION] OF ST_STATION_CTRL;
  _stStationState     : REFERENCE TO ARRAY[1..MAX_STATION] OF ST_STATION_STATE;

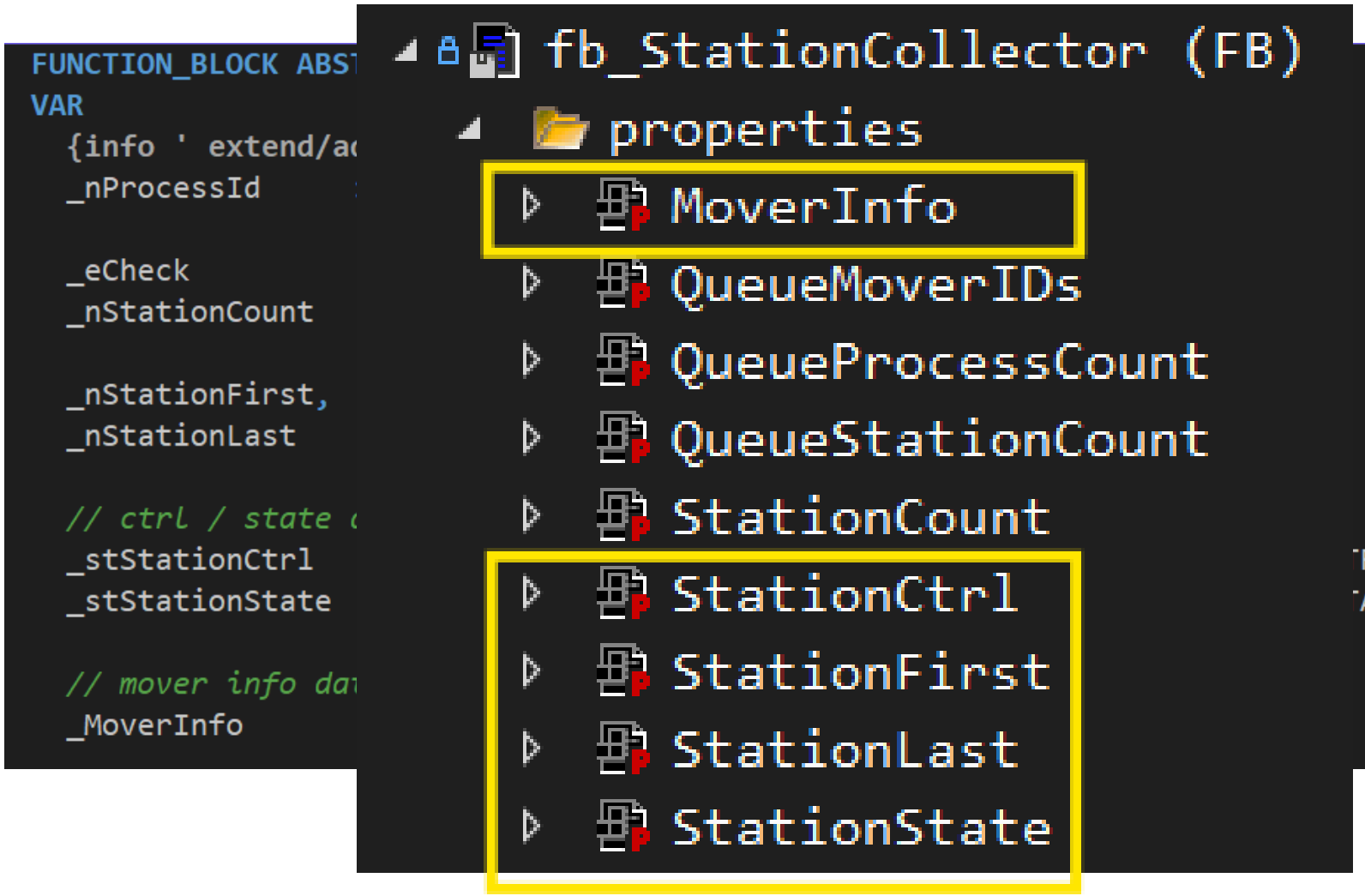
  // mover info datafield
  _MoverInfo          : REFERENCE TO ARRAY[1..MAX_MOVER] OF ST_MOVER_INFO;
```

- are connected from the outside (e.g. the stations).

BECKHOFF

- **The Collectors - basics**

- **StationCollector:**
  - The base class:
    - StationCtrl/StationState references to connect to base layer of stations
    - StationFirst/StationLast to define range of stations that are controlled
    - MoverInfo reference to all mover info data from the base layer



BECKHOFF

- **The Collectors - basics**

- **ProcessCollector:**
  - The extension:
    - Properties

```
//-----------------------------------------------------------------------------------------------
FUNCTION_BLOCK fb_ProcessCollector EXTENDS fb_StationCollector IMPLEMENTS I_ProcessCollector
VAR

  _stControl      : REFERENCE TO ST_PROCESS_CTRL;  // ctrl via property
  _stState        : REFERENCE TO ST_PROCESS_STATE; // state via property

  _eCmd,                                  // command for controlling the XtsStations
  _eCmdOld        : E_PROCESS_CTRL;       // logging of command on change
  _eStateProgress : E_PROGRESS;           // progress sub state for process
  _eResult        : E_PROGRESS;           // progress result for methods

  // ctrl words for XtsStations
  {attribute 'displaymode':='bin'}
  _wActivateStation     : T_PROCESS;  // bits enable XtsStations in this process, that's what drives me

  // ctrl data for used target XtsStations in target process
  {attribute 'displaymode':='bin'}
  _wTargetMask          : ARRAY[1..SIZEOF(T_PROCESS)*8] OF BYTE;    // mask for multiple PosStop in target
  _rTargetOffset        : ARRAY[1..SIZEOF(T_PROCESS)*8] OF LREAL;   // dyn offset for mulriple PosStop in target

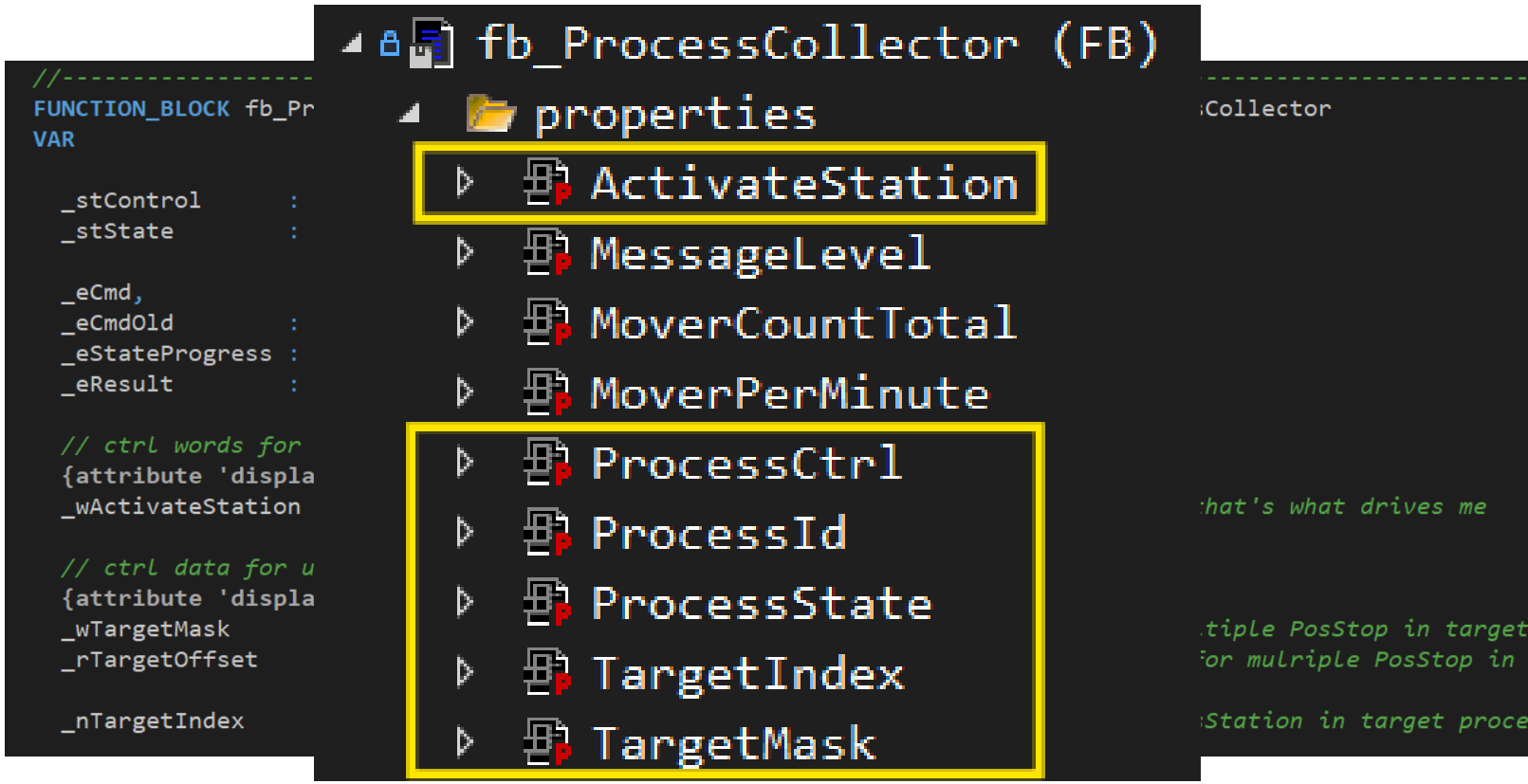
  _nTargetIndex         : ARRAY[1..SIZEOF(T_PROCESS)*8] OF USINT;   // index of XtsStation in target process
```

- are connected from the outside (e.g. the stations).

BECKHOFF

- **The Collectors**

- **ProcessCollector:**
  - The extension:
    - ProcessId(E_INSTANCE)
    - ProcessCtrl/ProcessState references to connect for commanding the collector
    - ActivateStation as bitmask; evaluated once on ENABLE
    - TargetIndex contains the individual targets for all stations in this collector's range.
    - TargetMask (optional bitmask for nests in stations)



LICENSE

BECKHOFF

XTS_TRANSPORT_LAYER project

MIT License
Copyright (c) 2026 HAUD

Permission is hereby granted, free of charge, to any person obtaining a copy of this software and associated documentation files (the "Software"), to deal in the Software without restriction, including without limitation the rights to use, copy, modify, merge, publish, distribute, sublicense, and/or sell copies of the Software, and to permit persons to whom the Software is furnished to do so, subject to the following conditions:

The above copyright notice and this permission notice shall be included in all copies or substantial portions of the Software.

THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS ORIMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THEAUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHERLIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN THESOFTWARE.